Systèmes

TD5 Administration de base sous Linux

Éditeurs de texte, utilisateurs et groupes, systemd et init, services et permissions

Objectifs

À la fin du TD, l'étudiant doit pouvoir :

- Créer et modifier un fichier avec Nano et Vim.
- Créer des utilisateurs et des groupes.
- Gérer l'appartenance aux groupes.
- Expliquer le rôle d'init et de systemd.
- Démarrer, arrêter et examiner un service.
- Lire et modifier les permissions et les propriétaires des fichiers.

Situation pratique

Vous administrez un poste Linux destiné à une équipe pédagogique. Vous devez créer deux comptes, préparer un espace partagé, rédiger une note et gérer un service.

Consignes :

- Exécuter les commandes une par une et vérifier leurs résultats.
- Conserver les commandes et observations dans un compte rendu.
- Ne pas arrêter les services réseau ou de connexion utilisés pour accéder à la machine.

Exercice 1 Les éditeurs de texte

1. Préparer les outils

```
sudo apt update
sudo apt install nano vim
```

Action : actualise les index des paquets puis installe les deux éditeurs.

Créer le dossier de travail :

```
mkdir -p ~/td_administration
cd ~/td_administration
```

2. Créer un document avec Nano

```
nano presentation.txt
```

Action : ouvre `presentation.txt` dans Nano. S'il n'existe pas, il sera créé lors de l'enregistrement.

Saisir :

```
Administration de base sous Linux
Nom :
Groupe :
Objectif : apprendre à administrer un poste Linux.
```

Raccourcis utiles de Nano

Raccourci	Action
`Ctrl+O`	Enregistrer, puis confirmer le nom avec Entrée
`Ctrl+X`	Quitter
`Ctrl+W`	Rechercher du texte
`Ctrl+K`	Couper la ligne courante
`Ctrl+U`	Coller le texte coupé
`Ctrl+G`	Afficher l'aide

Compléter le nom et le groupe, enregistrer puis quitter.

Vérifier :

```
cat presentation.txt
```

Action : affiche le contenu enregistré.

3. Modifier le document avec Vim

```
vim presentation.txt
```

Action : ouvre le fichier dans Vim.

Vim possède plusieurs modes :

Mode	Fonction
Normal	Naviguer et exécuter des actions
Insertion	Saisir du texte
Ligne de commande	Enregistrer, quitter ou lancer une commande

Effectuer les manipulations suivantes :

1. Appuyer sur `i` pour entrer en insertion.

2. Ajouter une ligne de commentaire.
3. Appuyer sur `Échap` pour revenir au mode normal.
4. Saisir `:wq`, puis Entrée, pour enregistrer et quitter.

Commandes utiles de Vim

Commande ou touche	Action
`i`	Passer en insertion avant le curseur
`Échap`	Revenir au mode normal
`:w`	Enregistrer
`:q`	Quitter si aucune modification non enregistrée ne bloque la sortie
`:wq`	Enregistrer et quitter
`:q!`	Quitter en abandonnant les modifications
`/Linux` puis Entrée	Rechercher `Linux`
`n`	Aller au résultat suivant
`dd`	Couper la ligne courante en mode normal
`p`	Coller après le curseur ou après la ligne, selon le texte coupé
`u`	Annuler la dernière modification

Exercice 2 Gestion des utilisateurs et des groupes

1. Identifier le compte courant

```
whoami
id
groups
```

Commande	Action
`whoami`	Affiche le nom de l'utilisateur courant
`id`	Affiche son UID, son GID principal et ses groupes
`groups`	Affiche ses appartenances aux groupes

UID : identifiant numérique d'un utilisateur.
GID : identifiant numérique d'un groupe.

2. Créer deux utilisateurs de test

```
sudo adduser td_alice
sudo adduser td_bob
```

Action : crée les comptes, leurs dossiers personnels et demande les informations nécessaires, notamment les mots de passe.

Sur Debian et Ubuntu, `adduser` propose une procédure interactive adaptée à cet exercice.

Vérifier :

```
getent passwd td_alice
getent passwd td_bob
```

Action : recherche les comptes dans les sources d'identités configurées.

Une ligne de compte contient notamment :

```
nom:x:UID:GID:commentaire:dossier_personnel:shell
```

Le caractère `x` ne représente pas le mot de passe : il indique généralement que les informations de mot de passe sont stockées séparément.

3. Créer un groupe de travail

```
sudo groupadd td_projet
```

Action : crée le groupe `td_projet`.

```
getent group td_projet
```

Action : affiche les informations du groupe.

4. Ajouter les utilisateurs au groupe

```
sudo usermod -aG td_projet td_alice
sudo usermod -aG td_projet td_bob
```

Élément	Action
`usermod`	Modifie un compte existant
`-a`	Ajoute aux groupes supplémentaires existants
`-G`	Indique les groupes supplémentaires
`td_projet`	Groupe à ajouter

Élément	Action
`td_alice` ou `td_bob`	Compte concerné

Attention : utiliser `-G` sans `-a` remplace la liste des groupes supplémentaires.

Vérifier :

```
id td_alice
id td_bob
```

Les sessions déjà ouvertes doivent être renouvelées pour prendre en compte les nouvelles appartenances.

5. Modifier un mot de passe

```
sudo passwd td_alice
```

Action : définit un nouveau mot de passe pour le compte de test.

Aucun caractère ne s'affiche pendant la saisie.

6. Ouvrir une session de test

```
su - td_alice
```

Action : ouvre un shell de connexion pour `td_alice` après authentification.

Vérifier :

```
whoami
id
pwd
```

Quitter cette session :

```
exit
```

7. Retirer un utilisateur d'un groupe

```
sudo gpasswd -d td_bob td_projet
```

Action : retire `td_bob` du groupe supplémentaire `td_projet`.

Vérifier :

```
id td_bob
```

Puis le réintégrer pour la suite :

```
sudo usermod -aG td_projet td_bob
```

Exercice 3 Comprendre init et systemd

1. Comprendre leur rôle

Au démarrage, le noyau lance un premier processus de l'espace utilisateur, généralement identifié par le **PID 1**. Ce processus participe à l'initialisation du système et à la gestion des autres processus.

Notion	Description
init	Nom traditionnel du premier processus ; désigne aussi des systèmes d'initialisation
SysV init	Organisation historique reposant notamment sur des scripts et des niveaux d'exécution
systemd	Système d'initialisation et gestionnaire de services fondé sur des unités
Service	Programme ou ensemble de processus fournissant une fonction
Unité systemd	Objet géré par systemd : service, cible, montage, minuterie, etc.
Cible ou target	Unité regroupant d'autres unités pour atteindre un état du système

2. Identifier le processus PID 1

```
ps -p 1 -o pid,comm,args
```

Action : affiche l'identifiant, le nom et la ligne de commande du processus PID 1.

Dans la machine virtuelle prévue pour ce TD, le résultat doit correspondre à systemd.

3. Examiner la cible par défaut

```
systemctl get-default
```

Action : affiche la cible que systemd cherche à atteindre par défaut au démarrage.

Exemples :

Cible	Signification générale
`multi-user.target`	Système multiutilisateur sans demander une session graphique
`graphical.target`	Système avec les services nécessaires à un environnement graphique

4. Examiner les cibles actives

```
systemctl list-units --type=target --state=active
```

Action : liste les unités de type cible actuellement actives.

Repère historique

Avec SysV init, on rencontre des scripts sous `/etc/init.d/` et des niveaux d'exécution. Avec systemd, la commande principale de gestion est `systemctl`.

Les commandes `init 0` et `init 6` sont traditionnellement associées à l'arrêt et au redémarrage. **Ne pas les exécuter pendant cet exercice.**

Exercice 4 Gestion des services

Nous utiliserons un service de test qui ne remplace aucun service système.

1. Créer un script simple

```
sudo nano /usr/local/bin/td-service.sh
```

Saisir :

```
#!/bin/sh

while true
do
    echo "Le service TD fonctionne"
    sleep 15
done
```

Action du script : écrit un message toutes les quinze secondes.

Enregistrer, puis rendre le script exécutable :

```
sudo chmod 755 /usr/local/bin/td-service.sh
```

Action : autorise le propriétaire à modifier le script et permet à tous de le lire et de l'exécuter.

2. Créer une unité systemd

```
sudo nano /etc/systemd/system/td-demo.service
```

Saisir :

```
[Unit]
Description=Service de demonstration du TD

[Service]
Type=simple
ExecStart=/usr/local/bin/td-service.sh
User=nobody
Group=nogroup
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Ce modèle est destiné à Debian et Ubuntu, où les identités `nobody` et `nogroup` sont normalement disponibles. Les vérifier si nécessaire avec :

```
getent passwd nobody
getent group nogroup
```

Comprendre l'unité

Directive	Action
`Description`	Décrit le service
`Type=simple`	Le processus lancé par `ExecStart` est le processus principal
`ExecStart`	Indique le programme à exécuter
`User` et `Group`	Définissent l'identité d'exécution
`Restart=on-failure`	Prévoit un redémarrage après certaines défaillances
`WantedBy`	Indique la cible à laquelle rattacher le service lors de son activation

3. Relire les fichiers d'unités

```
sudo systemctl daemon-reload
```

Action : demande à systemd de relire les définitions d'unités. Cette commande ne démarre pas le service.

4. Démarrer le service

```
sudo systemctl start td-demo.service
```

Action : démarre le service maintenant.

5. Examiner son état

```
systemctl status td-demo.service
```

Action : affiche l'état du service et quelques messages récents.

Appuyer sur `q` pour quitter le lecteur si nécessaire.

```
systemctl is-active td-demo.service
```

Action : indique si le service est actif.

6. Lire les journaux

```
sudo journalctl -u td-demo.service -n 10 --no-pager
```

Action : affiche les dix derniers messages disponibles pour ce service, sans lecteur interactif.

```
sudo journalctl -u td-demo.service -f
```

Action : suit les nouveaux messages. Appuyer sur `Ctrl+C` pour arrêter le suivi ; cela n'arrête pas le service.

7. Activer au démarrage

```
sudo systemctl enable td-demo.service
```

Action : configure le démarrage automatique du service selon la section `[Install]`. Cela ne le démarre pas immédiatement.

Vérifier :

```
systemctl is-enabled td-demo.service
```

8. Redémarrer, arrêter et désactiver

```
sudo systemctl restart td-demo.service
```

Action : arrête puis redémarre le service.

```
sudo systemctl stop td-demo.service
```

Action : arrête le service maintenant.

```
sudo systemctl disable td-demo.service
```

Action : retire sa configuration de démarrage automatique, sans nécessairement arrêter un service encore actif.

Tableau récapitulatif

Commande	Effet immédiat	Effet sur le démarrage automatique
`start`	Démarre	Aucun
`stop`	Arrête	Aucun
`restart`	Redémarre	Aucun
`enable`	Ne démarre pas à elle seule	Active
`disable`	N'arrête pas à elle seule	Désactive
`enable --now`	Démarre	Active
`disable --now`	Arrête	Désactive

Exercice 5 Permissions et espace partagé

1. Lire les permissions

```
ls -lh ~/td_administration/presentation.txt
```

Exemple de permissions :

```
-rw-r--r--
```

Position	Signification
Premier caractère	Type : `-` pour un fichier ordinaire, `d` pour un répertoire
Trois caractères suivants	Droits du propriétaire
Trois suivants	Droits du groupe
Trois derniers	Droits des autres utilisateurs

Signification des droits

Droit	Valeur	Pour un fichier	Pour un répertoire
`r`	4	Lire le contenu	Lister les noms des éléments
`w`	2	Modifier le contenu	Ajouter, supprimer ou renommer des éléments, avec les droits de traversée nécessaires
`x`	1	Exécuter	Traverser le répertoire et accéder à ses éléments connus

La suppression d'un fichier dépend principalement des droits sur son **répertoire parent**, pas du seul droit d'écriture sur le fichier.

2. Utiliser la notation numérique

```
chmod 600 ~/td_administration/presentation.txt
```

Action : donne lecture et écriture au propriétaire uniquement.

Mode	Permissions	Interprétation
`600`	`rw-------`	Propriétaire seul
`640`	`rw-r-----`	Propriétaire en lecture-écriture, groupe en lecture
`644`	`rw-r--r--`	Propriétaire en lecture-écriture, autres en lecture
`750`	`rwxr-x---`	Propriétaire complet, groupe en lecture-exécution
`755`	`rwxr-xr-x`	Propriétaire complet, autres en lecture-exécution

3. Utiliser la notation symbolique

```
chmod g+r ~/td_administration/presentation.txt
```

Action : ajoute la lecture pour le groupe.

```
chmod o-rwx ~/td_administration/presentation.txt
```

Action : retire tous les droits aux autres.

Symbole	Signification
`u`	Propriétaire
`g`	Groupe
`o`	Autres
`a`	Tous
`+`	Ajouter un droit
`-`	Retirer un droit
`=`	Fixer les droits indiqués

4. Créer un dossier partagé

```
sudo mkdir -p /srv/td_projet
sudo chown root:td_projet /srv/td_projet
sudo chmod 2770 /srv/td_projet
```

Commande	Action
`mkdir -p`	Crée le dossier
`chown root:td_projet`	Fixe le propriétaire à `root` et le groupe à `td_projet`
`chmod 2770`	Donne tous les droits au propriétaire et au groupe, aucun aux autres, et active le bit setgid

Le bit **setgid** sur un répertoire fait hériter aux nouveaux éléments le groupe du répertoire. Il ne garantit pas, à lui seul, que les fichiers seront modifiables par le groupe.

Vérifier :

```
ls -ld /srv/td_projet
```

L'option `-d` affiche les informations du répertoire lui-même.

5. Créer une note avec Alice

```
su - td_alice
```

Puis :

```
umask 0007
printf '%s\n' "Note partagee par Alice" > /srv/td_projet/note.txt
ls -l /srv/td_projet/note.txt
exit
```

Action de `umask 0007` : définit, pour cette session, les droits à retirer lors de la création de nouveaux éléments. Pour ce fichier créé par redirection, les permissions attendues sont `660`, soit `rw-rw----`.

Le groupe attendu est `td_projet`, grâce au setgid du dossier.

6. Modifier la note avec Bob

```
su - td_bob
```

Puis :

```
cat /srv/td_projet/note.txt
printf '%s\n' "Ajout de Bob" >> /srv/td_projet/note.txt
cat /srv/td_projet/note.txt
exit
```

Résultat attendu : Bob peut lire et compléter la note grâce à son appartenance au groupe et aux droits du fichier.

7. Modifier le groupe ou le propriétaire

```
sudo chgrp td_projet /srv/td_projet/note.txt
```

Action : change uniquement le groupe propriétaire.

```
sudo chown td_alice:td_projet /srv/td_projet/note.txt
```

Action : fixe à la fois le propriétaire et le groupe.

TP

Exercice 1 Les éditeurs de texte

Questions

1. Quelle différence existe entre Nano et Vim ?
2. Pourquoi faut-il revenir au mode normal avant de saisir `:wq` ?
3. Quelle commande permet de quitter Vim sans enregistrer ?

NB : pour répondre aux questions, il faut **des captures d'écran à l'appui des réponses**.

Exercice 2 Gestion des utilisateurs et des groupes

Questions

1. Quelle différence existe entre utilisateur et groupe ?
2. Quelle différence existe entre groupe principal et groupes supplémentaires ?
3. Pourquoi utiliser `usermod -aG` ?
4. Pourquoi une nouvelle session peut-elle être nécessaire après une modification des groupes ?

NB : pour répondre aux questions, il faut **des captures d'écran à l'appui des réponses**.

Exercice 3 Comprendre init et systemd

Questions

1. Quel processus possède le PID 1 sur votre machine ?
2. Quelle différence existe entre une unité `.service` et une unité `.target` ?
3. Quelle est la cible par défaut du système ?

NB : pour répondre aux questions, il faut **des captures d'écran à l'appui des réponses**.

Exercice 4 Gestion des services

Questions

1. Quelle différence existe entre `start` et `enable` ?
2. Pourquoi exécuter `daemon-reload` après avoir modifié une unité ?
3. Comment consulter les journaux d'un service ?
4. Pourquoi arrêter le suivi des journaux ne stoppe-t-il pas le service ?

NB : pour répondre aux questions, il faut **des captures d'écran à l'appui des réponses**.

Exercice 5 Permissions et espace partagé

Questions

1. Que signifie `chmod 640 fichier` ?
2. Quelle différence existe entre `chown` et `chgrp` ?
3. À quoi sert `x` sur un répertoire ?
4. Quel est le rôle du bit setgid ?
5. Pourquoi faut-il examiner aussi les droits du répertoire parent ?

NB : pour répondre aux questions, il faut **des captures d'écran à l'appui des réponses**.

Structure du rapport de TD (au format PDF uniquement)

Page de couverture

Nom de l'université

Faculté : [Nom de la faculté]

TD N°X – Systèmes

Nom & Prénom : .....

Niveau : .....

Date : .....

Description des résultats de la tâche

- ☑ Expliquer en quelques lignes ce que tu devais faire (objectif du TD).
- ☑ Décrire la démarche suivie.

Résultats de l'exécution des commandes

- ☑ Insérer des captures d'écran (les coller dans ton rapport).
- ☑ Ajouter une courte explication sous chaque capture pour montrer ce que la commande a produit.

Conclusions

- [x] Résumer ce que tu as appris.
- [x] Indiquer si la tâche est réussie, difficultés rencontrées et solutions.

Hébergement sur GitHub (via Git ou PowerShell)

Rapport de travail au format PDF uniquement

- Le rapport doit être rédigé puis exporté en .pdf (pas en Word, pas en .docx).

Hébergement GitHub obligatoire

- Hébergez sur GitHub :
- Le rapport en PDF.
- Le fichier/dossier qui contient les images (captures d'écran).

NB: Outil à utiliser pour l'hébergement
Vous devez obligatoirement utiliser Git ou Windows PowerShell pour :

- Initialiser le dépôt.
- Ajouter les fichiers.
- Faire un commit.
- Pousser (git push) sur GitHub.